Computer Graphics: Lecture 3

Getting Started

Welcome Back!

Last lecture we learned a bit of historical context. [Can anyone recall?]

Very importantly, we also learned about the *3D Graphics Pipeline*. [What is it and what are its stages?]

This lecture

This lecture we are going to get set up.

We will install the software we need to program 3D graphics applications.

But first, let's talk about how 3D graphics applications run.

3D hardware and software

Any modern computer for consumers has GPU support. There are two forms that can take:

1. Integrated graphics: your CPU is on the same chip as a GPU.
2. Discrete graphics: you have a separate GPU device (e.g., a graphics card)

For this class, it actually doesn't matter which you have. We aren't exactly going to be maxing it out (although you might in your term projects).

3D hardware and software (2)

There are several companies that make GPUs.

In the old days, companies like these created their own APIs for their own custom GPUs. Rendition cards used the Speedy3D API, Voodoo cards used Glide. Game consoles still often work this way.

However, Microsoft pushed hard to create a unifying API called Direct3D that any 3D card could support, allowing developers to write their renderer just for Direct3D and support any graphics card.

At the same time, major graphics innovator SGI released their custom API, IrisGL, under the name OpenGL. Unlike Direct3D, it was cross platform.

How these APIs work

When a 3D company makes a GPU, they develop *drivers* for it for the different platforms.

The platforms develop software *libraries* for the drivers. These libraries generally target C for mass compatibility (practically every language can call C functions).

The software links to the libraries and calls functions to draw lists of triangles and compile and use shaders.

Those functions end up calling the drivers, which interact with the custom firmware on the chip at a low level, to draw triangles into buffers.

3D hardware and software (3)

OpenGL's last update was in 2017. It's no longer adding new features.

Instead, it was superceded by an API called Vulkan, which is maintained by the same consortium (Khronos Group).

At the same time, Microsoft has continued to enhance Direct3D, and its current version is DirectX 12.

Not to be outdone, Apple has their own, called Metal, which is very popular with its users, but only works on Apple platforms.

3D hardware and software (4)

For the web, WebGL is a web version of OpenGL.

WebGPU is its replacement. WebGPU also works on desktop.

Behind the scenes, it is a thin wrapper that uses whichever API is most natural for your system:

- On a Windows system, it's typical for WebGPU to use DX12
- On a Mac, it's more likely to use Metal
- On Linux or otherwise, expect it to use Vulkan.

Vulkan is also not restricted to Linux, and you might get better results with it on a Windows or Mac system depending on the driver.

WebGPU is thin

I want to emphasize that WebGPU is a thin wrapper. If you go and learn Metal or something it will be very familiar.

It does not add much overhead at all.

Fundamentally, all of the so-called “modern” APIs are quite similar.

So don’t worry that you’re learning the wrong thing. WebGPU is just the simplest of the modern APIs, but everything it supports, Vulkan also supports.

How to make a 3D app

So, if you want to make a 3D program, you pick an API.

We're using WebGPU.

Then, you make sure that you link its library. We're writing code for the web, so this is done for us by the browser.

Finally, you make an application targetting that API. Typically this involves loading meshes, creating pipelines, and issuing draw calls to the API.

So, let's get started...

Questions?

Writing for the browser

We're going to write code that works in the browser.

The main scripting language for browsers is Javascript.

However, Javascript is a dynamically-typed language. We want static typing, so we will use Typescript. Typescript compiles into Javascript.

We also want to use some libraries. For things like Matrix math.

Therefore, we want a convenient package system that lets us install libraries, and bundle them in with our webpage.

That package system is the Node package manager, which you should already have installed.

Setting up a node project

For this class, all of your are going to create a single GitHub/Gitlab repo that will store all your class projects.

If you want, you can copy the `samples` directory in the repo these slides are stored in. It's already configured with the correct `package.json` and `tsconfig.json`.

However, I am going to give a very fast explanation of how the web and Node work, since many of you may not have written software this way before.

I want every bit of the system you build to be understandable.

HTML and Javascript

The web was intended to be a giant “hypermedia graph”.

Hypermedia is media that can link or reference other media.

The most common form of hypermedia is hypertext, which is basically text with links and other media mixed in with it. Web pages are hypertext.

The basic language that web pages are made in is called HTML: hypertext markup language. Markup means that the language is interspersed with the text that is contained on the page.

A basic web page

The “markup” part of HTML is made of **tags**.

Each tag has a name that says what kind of information it conveys about the page. Tags are contained within angle brackets. For example, `<p>` is a tag for marking a paragraph.

In order to define a paragraph, you start the text with `<p>`. Then you write the paragraph. Then, you *close* the tag by writing `</p>`.

Tags that refer to regions of text, like paragraphs, are always closed by preceding the name of the tag with a forward slash.

A basic web page (2)

Let's create a page called `index.html`. Just create an empty file with that name.

This is an important name. If someone navigates to a website without specifying which page they want, the browser will open `index.html` by default.

The browser will parse the HTML and *render* (which means *draw*) the webpage inside the window.

A basic web page (3)

Inside the file, put this:

```
<!DOCTYPE html>
<html>
    <head><title>My WebGPU sample</title></head>
    <body>
        <p>Hello.</p>
    </body>
</html>
```

Explaining the basic web page

The `<!DOCTYPE...` is a special tag called a “preamble”. If it’s not there, the browser won’t be able to assume the webpage is written in modern HTML. It will enable “quirks mode” to support Janky 90’s websites. We don’t want that.

The rest of the file is contained within `<html>` tags. Inside, there are two broad regions of information:

1. `<head>`, contains information about the page
2. `<body>`, contains the visible content of the page

In the `<head>` we set the `<title>`, which will be the name of the tab.
In the `<body>`, we write a single paragraph with `<p>`.

Questions?

Adding dynamic behavior

Not long after the web was invented, people wanted a way to make the sites do things, instead of just being static content that sat there.

There were several programming languages that were developed to do this, but the "winner" was Javascript.

Let's add some simple javascript. Modify your `<head>` as follows:

```
<head>
        <title>My WebGPU sample</title>
        <script type="module">
            console.log("hello, world");
        </script>
</head>
```

Adding dynamic behavior (2)

We added a `<script>` tag, which contains code written in another language (Javascript by default).

Your browser has native support for that language. It will compile and execute that code in a virtual machine that is bundled with it.

`type="module"` is an attribute. It's a way of customizing a tag. In this case, using `type="module"` adds more features to the javascript:

- It allows `import` to be used to bring in outside libraries (we will do this soon).
- It allows asynchronous code to be run (WebGPU is asynchronous)
- It ensures that the browser waits for the whole page to be loaded.

Introducing Javascript

Javascript is a basic imperative language.

It is named after Java, which it resembles, but it is a different language with different rules.

Some of you took object-oriented design with me, and already know Typescript, which is a super set of Javascript.

If you didn't, don't worry. Javascript is similar to other languages you know. If you know a scripting language like Python, and an application language like Java or C♯, you will be able to pick it up quickly.

Here, we're calling `log` on the built-in global console object.

Running the Javascript

Open your `html` file in a browser. The easiest way is to double-click it. (later we will use webservers, but let's keep it simple for now).

You won't see anything special or different. To view the "hello world" text that was just printed, you need to view the console.

On major browsers, the shortcut to it is `ctrl + shift + J`.

You should see that "hello world" was printed there.

There is also a `console.warn` method for warnings, `console.error` for errors, and `console.assert` for asserts.

Questions?

Node

Javascript originally only ran in the browser. If you wanted to distribute a Javascript program, you would bundle it in a web page.

However, people also wanted to be able to use Javascript as a general scripting language without needing to bundle an entire browser. This would enable *Full Stack* development, where one language could be used for both the web page and also the backend code.

This is what motivated the development of Node.

Node is a *runtime environment* for Javascript. A runtime environment is basically software that can run programs for a given language.

Runtimes

Most languages have a runtime. If you want to make a programming language, one of the main things you have to do is make a runtime.

There are some languages that don't require runtimes, like C and Rust, but scripting languages usually require runtimes.

Node is therefore software that lets you run Javascript code without needing a browser. You can say `node myprog.js` and it will execute the program without it needing to be bundled inside HTML.

I thought we were coding for the web?

Therefore, we actually could have done this class without Node. We actually will be using the browser.

But node is more than just a runtime. It comes with a package manager called NPM. NPM is perhaps the most popular package manager.

We are going to use NPM to install the libraries and other software we need. In particular: Typescript is distributed as an NPM package.

So here is what we must do next:

- Create a new NPM package of our own (to install packages inside of)
- Install the Typescript compiler package

Creating an NPM package

In order for a directory to be considered by NPM as package, it must contain a file in it called `package.json`

`json` stands for "Javascript Object Notation". It is how *objects* (basically dictionaries) are specified in Javascript. You can probably understand it without training, because it follows the curly-brace convention for describing hierarchical data.

The easiest way to create a `package.json` file is to run `npm init` in your console, which will ask you some questions...

Creating an NPM package (2)

The default options are fine with some exceptions:

- The main entrypoint is going to be `index.html`
- Make sure the "type" is set to `module`. Not `commonjs`.
- For license, I used the MIT license, but you can use "UNLICENSED" if you want all rights reserved. The default license is kind of weird.

After entering all the options, you should see a file called `package.json`. Go ahead and look at its contents: this describes the data

Testing NPM

Once the `package.json` file is created, execute `npm update`.

This will cause `npm` to search through your `package.json`, ensure it is valid, and update any packages you depend on.

We don't depend on any yet, so this is a way to make sure your directory is set up correctly.

After running that command, you should have 3 files in your directory: `index.html`, `package.json`, and `package-lock.json` (which contains the versions of the packages you depend on–so it won't have anything interesting in it yet).

Installing Typescript

Now, let's install Typescript.

Run: `npm install typescript`.

Now, you have typescript installed. This installed `tsc`, the typescript compiler, which you can now run on typescript files.

(Warning: don't run `npm install tsc`, that will install the wrong package, you want the `typescript` package)

`tsc` runs with a default configuration, but we need to modify it so that it works correctly with the libraries we will install. Let's do that now.

Configuring Typescript

To set up a Typescript configuration, run `npx tsc --init`

In the same way that NPM looks for `package.json` to know if a directory is an NPM package, Typescript looks for a file called `tsconfig.json` to know if the directory is a Typescript project.

We need to modify the `tsconfig.json` file that was just created.

First, insert this line right below the "target" line:

```
"lib": ["DOM"],
```

This will add support for the DOM, something I will explain in a bit.

Configuring Typescript (2)

Make sure that these lines are correct:

```
"module": "nodenext",
"target": "esnext",
```

These have to do with how modules are handled. Our code is using new module features (Javascript for the browser didn't used to support `import` and `export`, you had to manually include the files you needed in the right order).

Now, let's write some Typescript and compile it to make sure everything works.

Writing Typescript

Create a directory called `sample01` in your project directory.

Inside that directory, create a typescript source file named `sample.ts`.

Now, open up that file in your text editor and let's create a function:

```
export function alertFail(err: Error): never {
    alert(err);
    throw err;
}
```

Compiling Typescript

This code defines a function named `alertFail`. This function will take an `Error`, and pop it up in an `alert` (an error box). Then, it will throw the error. `never` means it does not return (it throws instead).

`export` means the function will be available to be `imported`

The types come after the variables in Typescript. `err: Error` means the parameter `err` has type `Error`, which is a simple class that stores error information.

Let's compile it, execute `npx tsc` in the root directory of your project (the root directory contains `package.json` and `tsconfig.json`).

Compiling Typescript (2)

`npx` stands for “npm execute”. It is a command that lets you execute programs inside of npm packages.

Here, we are executing `tsc`, which is the typescript compiler.

`tsc` will check to see if there is a `tsconfig.json` file. If it finds one (and it should), it will know that it is inside a typescript project.

It will then compile your uncompiled (or not-recently-enough-compiled) typescript files, including `sample01/sample.ts`.

The compiled Typescript

After running it, you should see `sample01/sample.js` has been created. This is your compiled javascript that was created from the typescript.

Go ahead an open it. It's just the typescript with the types removed. So now it says `err` instead of `err: Error`, and there's no more `: never`.

Typescript just gives us an extra net to catch type errors. For example, if we accidentally treat a string as a number, it will give us a compile error. However, if we pass those checks, it just writes out plain Javascript code.

Using the function

Let's use this function. Insert this code as the first line of the region inside your `<script type="module">...</script>` area in `index.html`:

```
import { alertFail } from "./sample01/sample.js"
```

Notice that we're importing a javascript file, not a typescript file. The browser does not know how to execute Typescript, so we compiled it.

The `./` is significant. If you leave that out, it won't work.

The import command returns a Javascript object normally. We use `{ alertFail } from` to pattern match on the object and extract the `alertFail` function out of it.[1]

[1] this follows basically the same rules as record pattern matching in the Haskell class some of you took with me

Using the function (2)

Now let's actually call the function. In the `script` area, add this:

```
alertFail(new Error("Oh no!"));
```

Your final script block should look like this:

```
<script type="module">
    import { alertFail } from "./sample01/sample.js"

    console.log("hello, world");
    alertFail(new Error("Oh no!"));
</script>
```

Starting a webserver

Modern browsers are sandboxed. They won't let you load files from the user's harddisk to prevent you from spying on them.

Therefore, we need to start a webserver. The server will load the files and then transfer them to the browser.

NPM has a simple server available: `npm install -g serve`

Be sure to use the `-g` to install it globally. It has lots of dependencies that will bloat your repository, and we don't want to worry about vulnerability notices.

Starting a webserver (2)

Run the webserver like this: `npx serve`

You might want to use another terminal, it blocks the terminal as long as it's running.

By default, your webserver will run on port 3000.

Open your web browser and go to that port by entering this address: `localhost:3000` (replace 3000 with the other port if the server starts on another port: it will tell you which port in the command line)

Localhost refers to your own computer, which the server is running on.

Using the function (3)

When you navigate to `localhost:3000`, your browser will ask the webserver for `index.html` automatically.

Open the page, and expect a message box to pop up. This is the alert message we defined earlier.

Questions?

Especially if you're following along...

Let's make a WebGPU program

If everything works, that means your basic workflow is:

- Write the Typescript sample code
- Compile with `npx tsc`
- Refresh the page

You can actually have it compile automatically by running:

```
tsc --watch
```

This will load the compiler in watch mode. If you save a `.ts` file, it will automatically recompile it.

There's one more thing we want to do first for WebGPU support...

Installing type definitions

Typescript is designed to work with any existing Javascript workflow.

For example, you can take someone's Javascript file that doesn't have static types, and add your own type declarations to it.

This also works for built-in javascript, such as WebGPU.

In your root directory, run `npm install @webgpu/types`

Packages that start with `@` mainly contain type definitions.

This will install a bunch of files that Typescript will automatically load to know about WebGPU, so you'll get better compiler hints.

Using the type definitions

Once they are installed, add this line to your `tsconfig.json` file, below the line that we added that said `"lib": ["DOM"]`:

```
"types": ["@webgpu/types"],
```

Now, the compiler and language server will know to check for WebGPU type definitions. This will be extremely helpful. Make sure there are no errors here.

Updating our HTML file

Modern HTML (HTML 5) supports a special tag: `<canvas>`.

A canvas is a rectangular region on the page that you can draw to.

Importantly: WebGPU can also draw to it.

Therefore, we need to create a canvas.

Inside the `<body>...</body>` region of your index.html, insert this:

```
<canvas width="800" height="600"></canvas>
```

This will add a `canvas` to your page, which is 800 by 600 pixels in dimension.

Let's make a WebGPU program

Now, let's write a function that will initialize WebGPU. We will have to understand a few types of objects:

- **Adapter**: represents an actual GPU. It can be a physical GPU, or a virtual GPU (like when you run in a virtual machine). You can inspect the available features in the adaptor object. If you have a discrete and integrated GPU, you will have at least 2 adaptors to choose from.
- **Device**: represents the GPU configured with the settings you are going to draw with and actual features you want to use (which might be less extreme). This object has most of the WebGPU methods.
- **CanvasContext**: represents a region we can draw to.

Let's make a WebGPU program (2)

Let's start our function. Here is its header:

```
export async function initWebGpu():
    Promise<[GPUDevice, GPUCanvasContext, HTMLCanvasElement]>
{ ... }
```

We already talked about `export`. `async` means that the function is asynchronous. That is, it may execute code in its own chain of execution.

WebGPU is an asynchronous API. It will frequently do long operations on their own thread of execution so that your page can do other things while it waits.

Let's make a WebGPU program (3)

```
export async function initWebGpu():
    Promise<[GPUDevice, GPUCanvasContext, HTMLCanvasElement]>
{ ... }
```

In Javascript, every asynchronous function returns a `Promise`.

A `Promise<String>` is not a string. It's an object that will eventually return a string when it is ready.[1]

Inside an `async` function, you can `await` a promise, which will wait for the value to become available.

---

[1]Students who took programming langauge design: yes, Promise is very similar to the IO monad. Its .then method is very similar to >>=.

Let's make a WebGPU program (4)

```
export async function initWebGpu():
    Promise<[GPUDevice, GPUCanvasContext]>  { ... }
```

In this case, the promise will have a **tuple** inside it. A tuple is like an array, but the values can have different types, and its length is known at compile time.

The promise we return will contain:

- The device, so we can actually draw things
- The canvas context, which contains things like the image format and a reference to the canvas itself (so we can get its width and height)

Let's make a WebGPU program (5)

Now, let's check for WebGPU support:

```
const gpu = navigator.gpu;
if (!gpu) {
    alertFail(new Error("Browser does not support WebGPU"));
}
```

`navigator` is a global object that represents the browser itself.

The `gpu` field will have the value `undefined` (kind of like `null`) if WebGPU is not supported. `!undefined` is `true`.[1]

[1] Javascript is odd in that it has both `null` and `undefined`. The difference is their types: `null` has type `Object`, and `undefined` has type `undefined`. In typescript, people prefer to use `undefined`, because it lets the type be more explicit.

`const`

Const is one keyword we can use to define variables in Javascript.

For example, `const x = 7;` creates a constant named x.

You aren't allowed to change constants to point to other values:

```
const x = 7;
x = 8; // this is an error`
```

However, you are allowed to change data *inside* a constant:

```
const array = [1, 2, 3]; //use brackets to define an array
array[0] = 3;
// now array is [3, 2, 3]
```

Let's make a WebGPU program (6)

Now, let's grab our canvas:

```
const canvas = document.querySelector("canvas");

if (!canvas) {
    alertFail(new Error("Web page does not have a canvas element."));
}
```

`document` is a global object that represents the webpage itself.

The webpage is stored as a tree, following what is called the "document object model" (DOM). This is why we added `lib: ["DOM"]` to `tsconfig.json`, so that it would know about things like `navigator` and `document`.

Let's make a WebGPU program (7)

`querySelector` searches through the DOM until it finds an element that matches the given query.

In our case the query `"canvas"` means "anything with the canvas tag".

If there are more than one, it returns the first one.

If there aren't any, it returns `undefined`. This will let you detect if you spelled your `canvas` tag wrong or forgot to include it.

Let's make a WebGPU program (8)

Now we need to get one of the user's adapters:

```
const adapter =
    await navigator.gpu.requestAdapter({powerPreference: "low-power"});

if (!adapter) {
    alertFail(new Error("No WebGPU compatible GPU device available."));
}
```

Here, we set the power preference to "low power". There's also a "high-performance" option.

Many systems have multiple adapters. These settings are more likely to select integrated graphics (which is fine for what we're doing here).

Configuration objects

Let's quickly mention what that `{...}` thing is: it's an object.

Objects in Javascript are basically dictionaries. They follow very similar rules to dictionaries in other languages, like Python.

The entries of the object are written `key: value`.

You access them with a dot. So we could have done this:

```
const adapterConfig = {powerPreference: "low-power"};
// oops, changed my mind vv
adapterConfig.powerPreference = "high-performance";
/*other stuff*/...gpu.requestAdapter(adapterConfig);
```

Configuration objects (2)

WebGPU loves using objects to store configuration data.

This is the main reason we are using Typescript. Because those configuration objects can have types. This will protect us from forgetting an important field, and it will let us view which fields we need to provide and what their types are.

This is how it works:

```
type SomeObj = {x: number, y: string};
const obj: SomeObj {x: 20, y: "hi"};
```

Here, the type of the object requires that we have a number named ‘x’ and string named ‘y’. If we forget one, or get the type wrong: error.

Types are stripped out when we compile. They are only for error checking.

Configuration objects (3)

There are tons of configuration objects in WebGPU

When we installed `@webgpu/types`, we brought in type definitions for all of them.

This means, if we call a method that uses a configuration object, we can use our text editor to tell us what needs to go in it.

For example: `...requestAdaptor({ |<- cursor })`

My mouse cursor is where the `|` is in VS Code. I type `Ctrl + Space`, this will pop up a window showing all the required fields, as well as optional fields (ending with '?'). This is a *huge* time saver. Make sure to use it!

Questions?

Let's make a WebGPU program (9)

The adapter has a `.info` field, which contains information about it. It also has fields that tell you things like how much memory it has, so you can inform a user if the adapter they want to use isn't beefy enough.

The main purpose of the adapter is to create a device. To do this, we request a device with the features we want (the defaults are fine):

```
const device = await adapter?.requestDevice({
    label: "Our basic WebGPU device", // optional label for error messages
    requiredFeatures: [], // we're good with the defaults
});

if (!device) {
    alertFail(new Error("unable to create a device."));
}
```

Let's make a WebGPU program (10)

```
const context = canvas.getContext('webgpu');

if (!context) {
    alertFail(new Error("unable to aquire a webgpu canvas context."));
}
```

We create a context that supports WebGPU so we can draw to the page.

It's also possible to use other APIs that draw to canvases, such as WebGL, or HTML 2D renderers.

Let's make a WebGPU program (11)

```
context.configure({
    device: device,
    format: gpu.getPreferredCanvasFormat(),
    alphaMode: "opaque", // we'll talk about this later
});
```

We need to call `.configure` on our context to tell it what device is going to draw pixels into it.[1]

Format will either be BGRA (on Windows) or RGBA on everything else. Windows likes Blue to be first.

[1] Note, even though `context` is a constant, we are allowed to call methods on it that change it.

Let's make a WebGPU program (12)

Finally, we need to return the device and context we created. We are going to use these objects in all our samples:

```
return [device, context];
```

Because our function is marked `async`, this tuple of 2 values will be wrapped in a `Promise` automatically.

Questions?

Let's render something

We don't just want to initialize WebGPU, we want to render something to know that it's working.

Let's draw a solid color, because that's the simplest thing to do.

Start by defining a function to render (each of our samples will have a function or method that renders the image: later they will be animated)

```
export function renderSample01(
    device: GPUDevice,
    context: GPUCanvasContext,
): void
{
  ...
}
```

Let's render something (2)

Inside that function, we need to create a **command encoder**.

Command encoders are a feature of modern graphics APIs.

They act like a scripting language compiler. We, don't send commands to the GPU right away. Instead, we batch them together to be executed all at once later (which is much faster)

```
const encoder = device.createCommandEncoder();
```

The most important kind of command is to perform a "render pass". A pass is basically when we select which data we want to be available, and then draw a batch of objects. Let's define a pass...

Let's render something (3)

```
const pass = encoder.beginRenderPass({
    colorAttachments: [
        {
            loadOp: "clear",
            storeOp: "store",
            view: context.getCurrentTexture(),
            clearValue: {r: 0.7, g: 0.8, b: 0.9, a: 1.0},
        }
    ]
});
```

A pass needs one of those descriptor objects. The only required field is named "colorAttachments".

Attachments

An attachment is a data buffer that is "attached" to a pass, to be used as the destination for rendering operations.

Attachments have several important fields to describe:

1. `loadOp` can either be "clear" or "load". Clear means that when the attachment is first loaded, it is cleared to a solid color.[1]
2. `storeOp` can either be "store" or "discard". "Discard" means the pixels are thrown away. This is useful when you want to write the results of the pass manually (i.e., to some storage in the GPU).

[1] This is actually surprisingly slow, and not required for indoor scenes, so "load" can be used to not do it. It's slow because of all the pixels that need to be filled in. "fill rate" is the speed at which pixels are filled in, and it's a major bottleneck for 3D graphics.

Attachments (2)

3. `view` is the image that we'll be writing to. In this case, the canvas.
4. Lastly, if `loadOp` is "clear", you can provide a clear color, which is an object with `r` for red, `g` for green, `b` for blue, and `a` for alpha. We'll discuss how colors work later, but for now, note that r = 0.7, g = 0.8, b = 0.9, a = 1.0 correspond to a nice light blue.

Colors like that blue are easy to remember, but they also make it easy to detect errors (we usually expect all white or all black in the case of an error).

Let's render something (4)

After defining our pass, we tell it the viewport to draw to, and tell it "okay that's it, you're done."

```
pass.setViewport(0, 0,
  context.canvas.width, context.canvas.height, 0, 1);
pass.end();
```

It automatically clears the attachment (the colors we're drawing) to the color we want, so we don't actually have to draw anything.

Let's render something (5)

Last but not least, we "finish" our encoder, telling it "okay, that's all the commands". The result of this is a compiled list of commands.

We then submit those commands to the GPU. They will be executed as soon as they can be.

```
const commands = encoder.finish();
device.queue.submit([commands]);
```

`[commands]` is an array of one object. We can submit a list of command buffers if we want to prepare multiple frames at once. We usually want to draw the next frame immediatelly though.

Let's integrate

Last but not least, let's actually call the render function to get our blue background.

First, make sure to compile the typescript with `npx tsc` and fix errors.

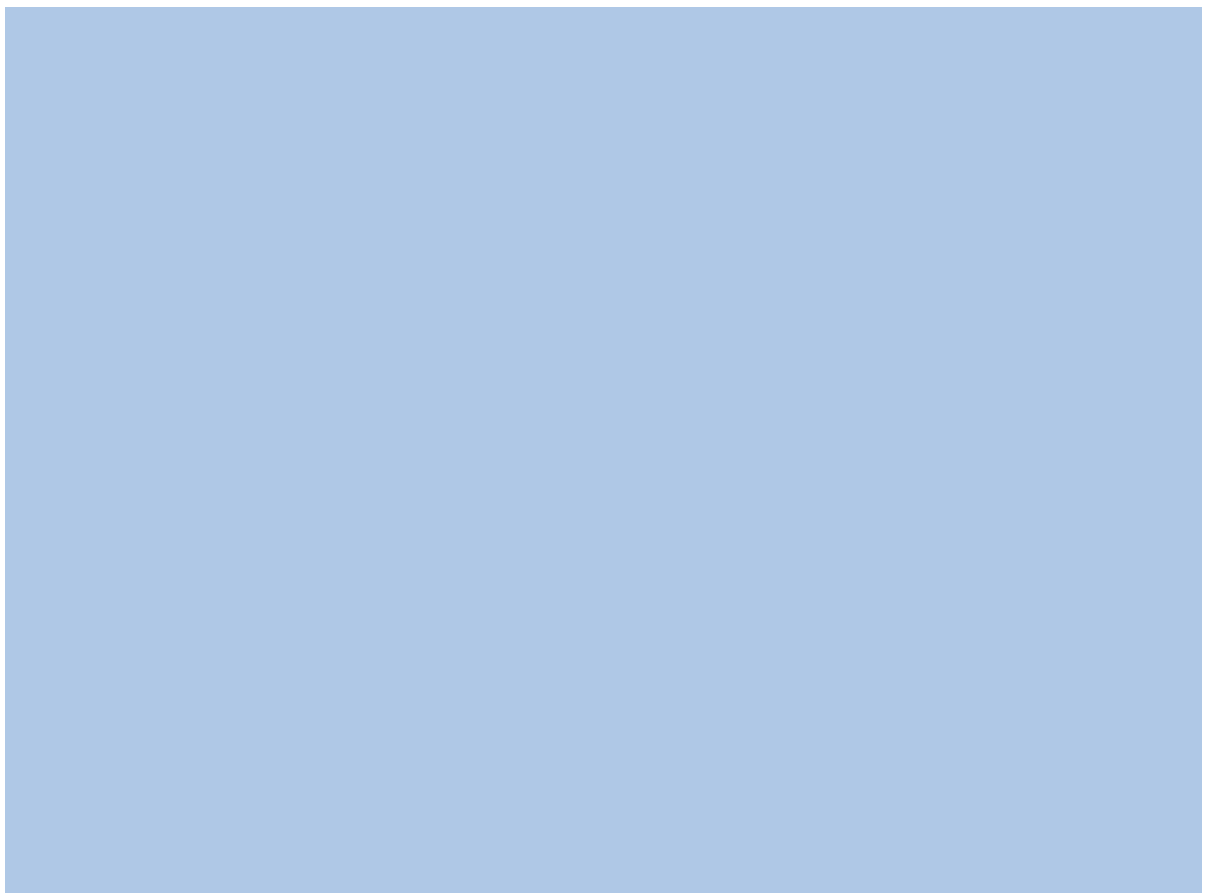
Then, update our imports:

```
import { alertFail, initWebGpu, renderSample01 } from "./sample01/sample.js"
```

And lastly, call the function after initializing:

```
const [device, context] = await initWebGpu();
renderSample01(device, context);
```

Behold



Questions?

Don’t worry

I know that was quite a whirlwind

Don’t worry, we can keep most of it next time. These lessons will build on each other, and most of the time the previous lesson can be built-on to build the next one.

However, I found that starting from scratch really helped me learn the API. I found I could remember the main steps I needed after a few samples.

So I recommend making sure you can do this from scratch (except with the help of your `ctrl + space` shortcut).

Next time

Next time it's the big one: we draw a *triangle*!

I hope you're stoked. I am!

Comprehension Exercises

- Change the color from light blue to a lime green to see if you can figure it out.
- Store the color in an object, and set its `r`, `g`, `b`, and `a` fields using the dot notation you're familiar with from other languages. You can write `const color = {}` to create a new empty object.
- Notice how we often write `alertFail(new Error("..."))`. Write a function called `alertStr` which takes a string, constructs the `Error`, and which calls `alertFail` with that error.
- Read this chapter from the book. Can you build the sample from scratch? The key is the device queue, the encoder, and the render pass. You can work backwards from there.